

Sistem za upravljanje skladistem

Modularni monolit - Clean i heksagonalna arhitektura

1. Opis sistema

Desktop aplikacija (WPF) za upravljanje skladistem, izgradjena kao modularni monolit. Svaki poslovni domen (proizvodi, kategorije, korisnici, nalozi, obavestenja) izdvojen je u sopstveni projekat (modul) sa jasno definisanim granicama i zavisnostima. Moduli komuniciraju preko interfejsa (portova), a konkretne implementacije (adapteri) nalaze se u zasebnom Infrastructure modulu. Podaci se cuvaju u JSON datotekama.

Funkcionalni zahtevi

- Prijava korisnika
- Kreiranje, izmena i pregled proizvoda
- Kreiranje i pregled kategorija
- Kreiranje naloga sa stavkama (slozenija operacija - kalkulacija i azuriranje stanja)
- Pregled naloga
- Automatsko obavestenje pri kreiranju naloga (dogadjaj)

Nefunkcionalni zahtevi

- Odrzivost - svaki modul se moze menjati nezavisno, bez uticaja na ostale module
- Testabilnost - aplikacioni sloj (servisi) testira se preko portova, bez zavisnosti od UI-a ili stvarne perzistencije
- Prosirivost - novi modul (npr. novi nacin izvoza) dodaje se implementacijom novog porta, bez izmene postojećeg koda

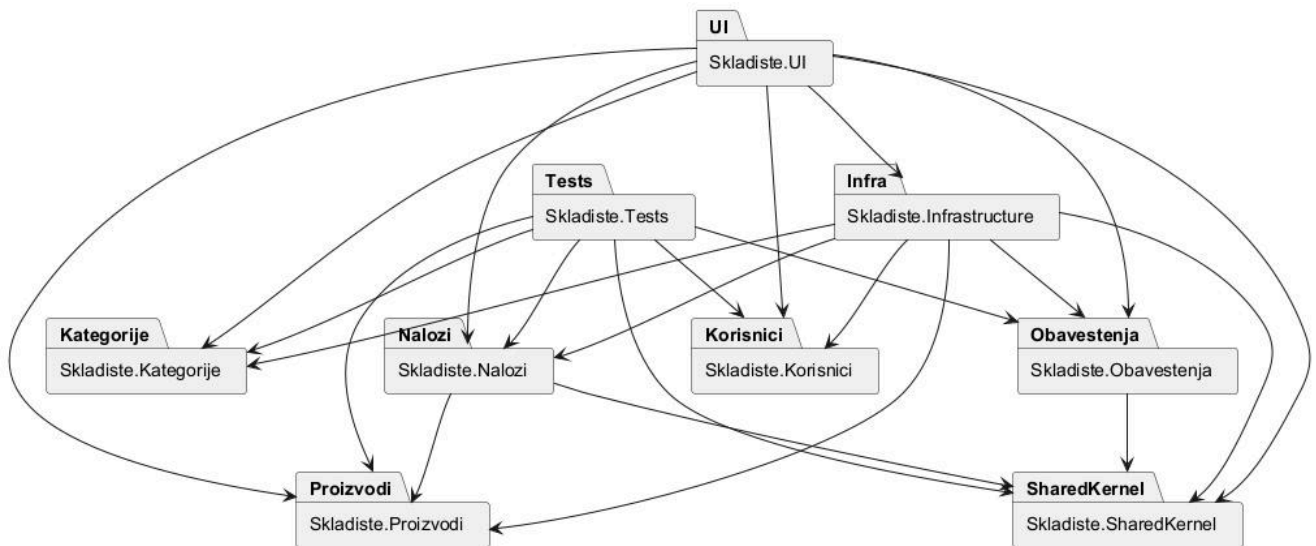
2. Use case-ovi

- Prijava korisnika - korisnik unosi korisnicko ime i lozinku; sistem proverava kredencijale.
- Kreiranje proizvoda - korisnik unosi podatke o proizvodu (naziv, sifra, cena, kolicina, kategorija).
- Izmena proizvoda - korisnik menja podatke postojeceg proizvoda.
- Pregled proizvoda i kategorija - korisnik pregleda spiskove proizvoda i kategorija.
- Kreiranje naloga sa stavkama - korisnik bira proizvode i kolicine; sistem izracunava ukupnu vrednost naloga i azurira kolicinu na stanju za svaki proizvod.
- Pregled naloga - korisnik pregleda spisak kreiranih naloga.
- Automatsko obavestenje pri kreiranju naloga - kada se nalog kreira, Nalozi modul objavljuje dogadjaj na koji Obavestenja modul reaguje i prikazuje poruku korisniku, bez direktne zavisnosti izmedju ta dva modula.

3. Dijagram modula

Aplikacija je podeljena u 9 modula (projekata): pet funkcionalnih modula (Proizvodi, Kategorije, Korisnici, Nalozi, Obavestjenja), SharedKernel (zajednicki domenski elementi bez ikakvih zavisnosti), Infrastructure (adapteri za sve portove), UI (WPF kompozicioni koren) i Tests (jedinicni testovi).

Granice modula su strogo po poslovnom domenu - svaki funkcionalni modul sadrzi sopstveni entitet, port (interfejs repozitorijuma) i aplikacioni servis. Zavisnosti idu iskljucivo u jednom smeru: UI i Infrastructure zavise od funkcionalnih modula (nikad obrnuto), Nalozi zavisi od Proizvodi (da bi azurirao stanje), a Obavestjenja i Nalozi zavise od SharedKernel (dogadjaj i IEventBus port). Nijedan modul ne zavisi kružno od drugog.



4. C4 dijagrami arhitekture

C4 model opisuje arhitekturu sistema kroz cetiri nivoa zumiranja: Context (sistem i njegovi korisnici), Container (izvršne/deployabilne jedinice sistema), Component (unutrasnja podela kontejnera na komponente) i Code (klase - ovaj nivo se ovde ne prikazuje jer je pokriven dijagramom klasa/modula). U nastavku su prikazana prva tri nivoa, sa detaljnim objasnjenjem svakog.

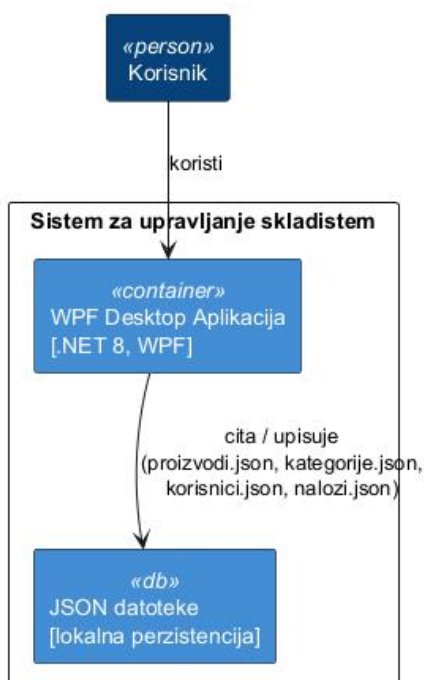
4.1 Context (Nivo 1)

Context dijagram prikazuje sistem kao jednu celinu i njegove korisnike, bez ikakvih tehnickih detalja. Ovde se vidi da postoji jedan tip korisnika (u ulozi Administratora ili Magacionera) koji koristi sistem za upravljanje skladistem u celini - prijavljuje se, upravlja proizvodima i kategorijama i kreira naloge.



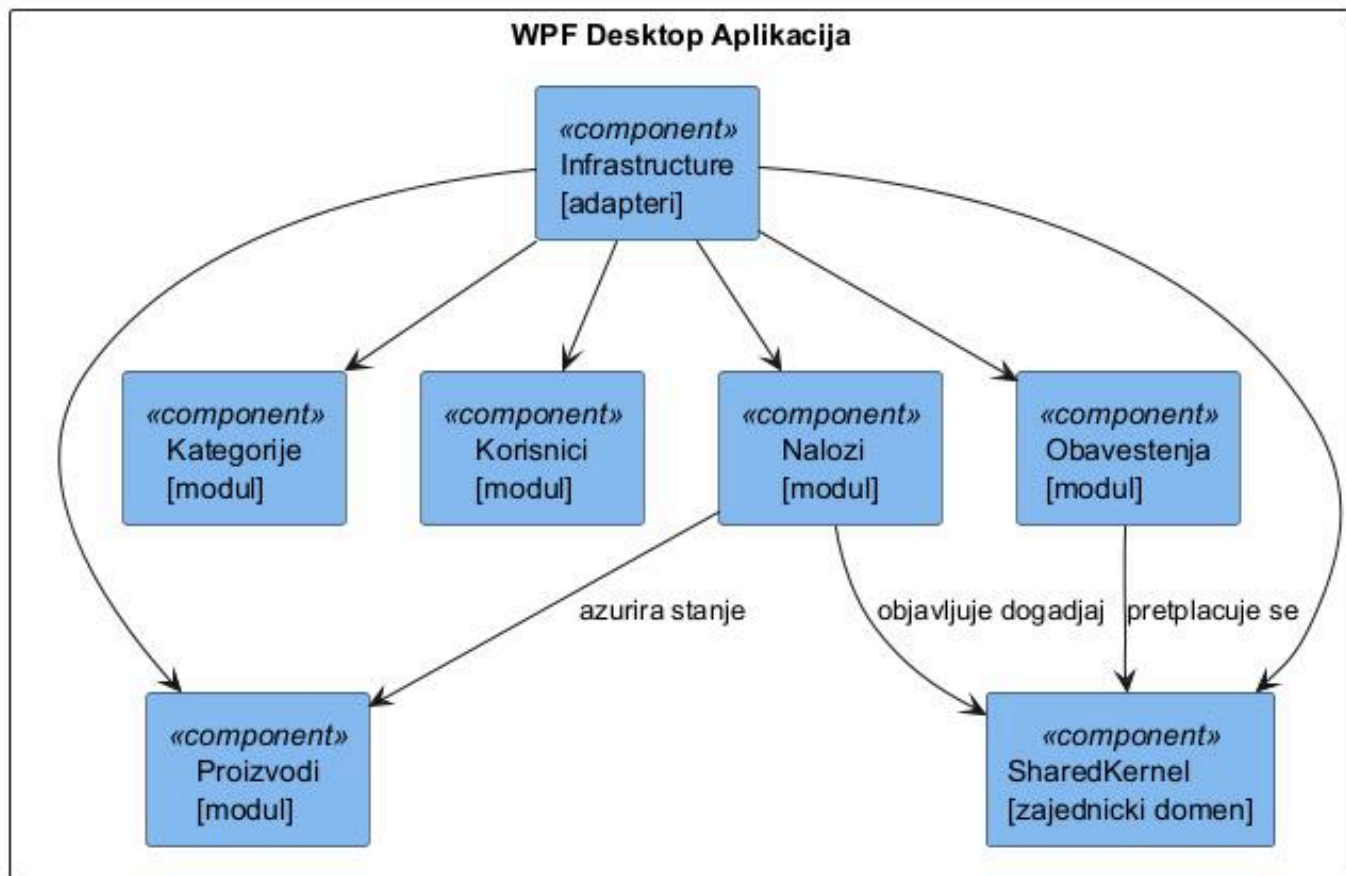
4.2 Container (Nivo 2)

Container dijagram deli sistem na izvršne jedinice koje se posebno pokrecu ili skladiste. U ovom sistemu postoje dva kontejnera: WPF Desktop Aplikacija (jedini izvršni program, .NET 8) i JSON datoteke (kontejner za perzistenciju - cetiri odvojene datoteke, po jedna za proizvode, kategorije, korisnike i naloge). Za razliku od Context nivoa, ovde se vec vidi da aplikacija cita i upisuje podatke u lokalne datoteke, umesto u udaljenu bazu ili servis.



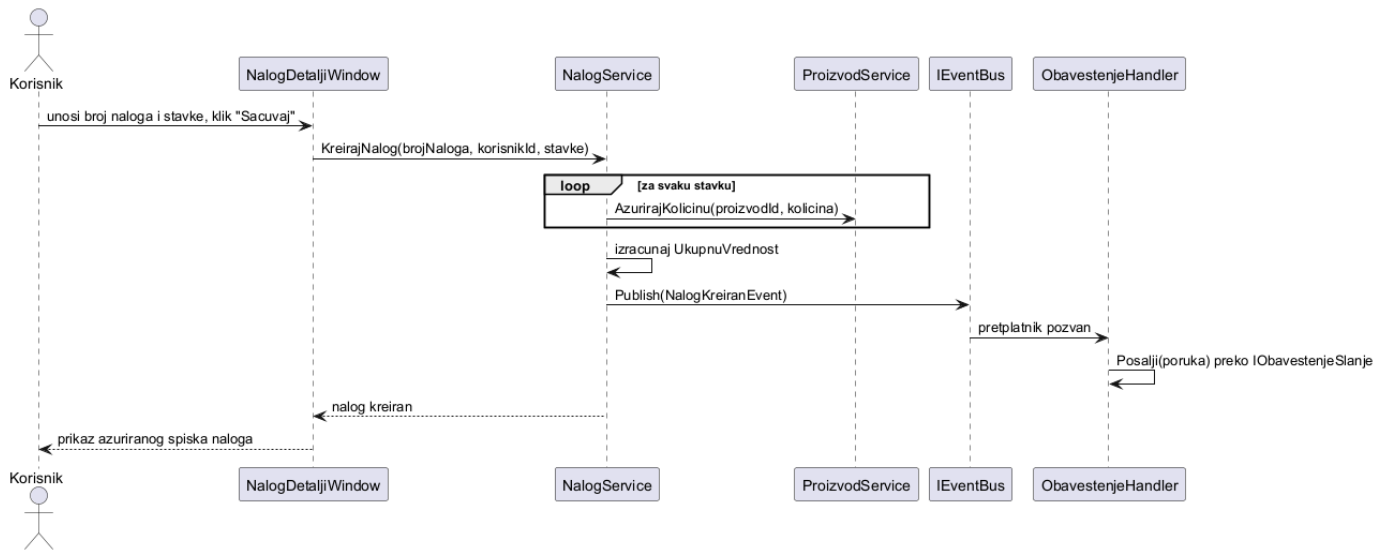
4.3 Component (Nivo 3)

Component dijagram otvara WPF Desktop Aplikaciju kontejner i prikazuje njegovu unutrašnju podelu na module - ovo je nivo na kome se modularni monolit direktno vidi. Svaki pravougaonik predstavlja jedan modul (projekat) iz resenja. Strelice pokazuju stvarne zavisnosti iz koda: Nalozi poziva Proizvodi da azurira stanje i objavljuje dogadjaj preko SharedKernel-a, Obavestenja se pretplacuje na taj dogadjaj preko SharedKernel-a, a Infrastructure implementira portove svih funkcionalnih modula (zato ima strelicu ka svakom od njih). Ovaj dijagram je direktna vizuelna potvrda da je zahtev za modularni monolit sa jasno definisanim, jednosmernim zavisnostima ispunjen.



5. Dijagram sekvenci

Prikazuje redosled poziva pri kreiranju naloga, ukljucujuci azuriranje stanja proizvoda i objavljivanje/ preuzimanje dogadjaja izmedju Nalozi i Obvestenja modula preko IEventBus porta.



6. Arhitektonske odluke

Zasto flat moduli (jedan projekat po modulu)

Svaki funkcionalni modul (Proizvodi, Kategorije, Korisnici, Nalozi, Obavestenja) drzi svoj entitet, port i aplikacioni servis u jednom projektu, bez dodatne podele na Domain/Application potprojekte. Za obim ovog sistema to je dovoljno da se zadovolji Clean Architecture princip odvajanja poslovne logike od infrastrukture - odvajanje se postize izmedju modula i Infrastructure projekta, a ne unutar svakog modula posebno.

Zasto JSON perzistencija

JSON datoteke su najjednostavniji nacin da se demonstrira razdvajanje portova (IProizvodRepository, IKategorijaRepository, IKorisnikRepository, INalogRepository) od konkretne implementacije (Infrastructure), bez potrebe za bazom podataka. Zamena JSON adaptera za, na primer, SQLite adapter ne bi zahtevala nikakvu izmenu u funkcionalnim modulima.

Zasto in-memory event bus

IEventBus (SharedKernel) i InMemoryEventBus (Infrastructure) su najjednostavniji nacin da se realizuje mehanizam dogadjaja unutar jednog procesa (modularni monolit, ne distribuirani sistem). NalogService objavljuje NalogKreiranEvent, a ObavestenjeHandler se pretplacuje na njega u konstruktoru - Nalozi modul pritom nema nikakvu direktnu zavisnost od Obavestenja modula.

Dependency Injection

Microsoft.Extensions.DependencyInjection kabljuje se na jednom mestu - u Skladiste.UI/App.xaml.cs (kompozicioni koren). Svi portovi, adapteri i servisi registruju se tamo, a MainWindow i LoginWindow dobijaju svoje ViewModel-e iskljucivo kroz konstruktor (bez direktnog 'new' poziva u code-behind-u), cime se izbegava tipicna greska da UI zaobidje DI kontejner.

7. Struktura projekta

Moduli

- Skladiste.SharedKernel - NalogKreiranEvent, IEventBus (bez zavisnosti)
- Skladiste.Proizvodi - Proizvod, IProizvodRepository, ProizvodService
- Skladiste.Kategorije - Kategorija, IKategorijaRepository, KategorijaService
- Skladiste.Korisnici - Korisnik, IKorisnikRepository, AutentifikacijaService
- Skladiste.Nalozi - Nalog, StavkaNaloga, INalogRepository, NalogService
- Skladiste.Obavestenja - IObavestenjeSlanje, ObavestenjeHandler
- Skladiste.Infrastructure - adapteri (JSON repozitorijumi, InMemoryEventBus, PopupObavestenjeSlanje)
- Skladiste.UI - WPF kompozicioni koren, ViewModeli, Views
- Skladiste.Tests - jedinичni testovi (Application sloj, fake portovi)

8. Kratak opis implementacije

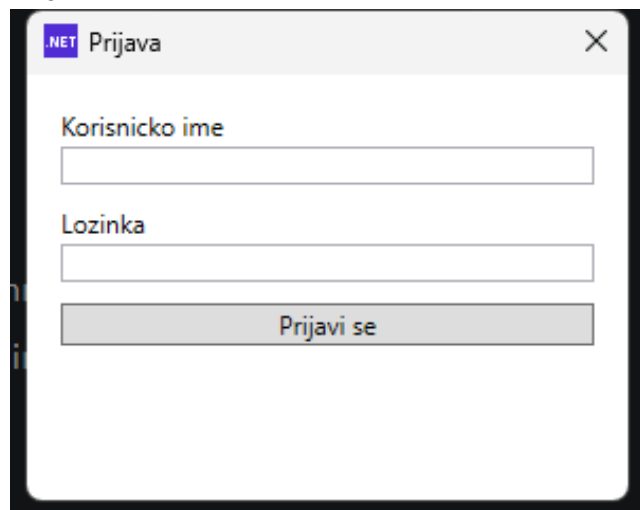
ViewModeli u Skladiste.UI koriste ICommand (RelayCommand) i INotifyPropertyChanged za binding. Dijalozi (npr. kreiranje naloga) direktno kreiraju svoj ViewModel jer zavise od trenutno izabranih podataka, dok MainWindow i LoginWindow dobijaju svoj ViewModel iz DI kontejnera. Kreiranje naloga (NalogService.KreirajNalog) je najsloženiji tok: za svaku stavku pronalazi proizvod, racuna cenu, azurira kolicinu na stanju preko ProizvodService, sabira ukupnu vrednost i na kraju objavljuje dogadjaj.

9. Testiranje

Skladiste.Tests sadrzi 6 jedinичnih testova nad ProizvodService i NalogService, bez ikakve UI ili JSON zavisnosti - koriste se rucno pisani fake portovi (FakeProizvodRepository, FakeNalogRepository, FakeEventBus) koji cuvaju podatke u memoriji. Testovi proveravaju kreiranje/izmenu/brisanje proizvoda, azuriranje kolicine, kalkulaciju ukupne vrednosti naloga i objavljivanje dogadjaja.

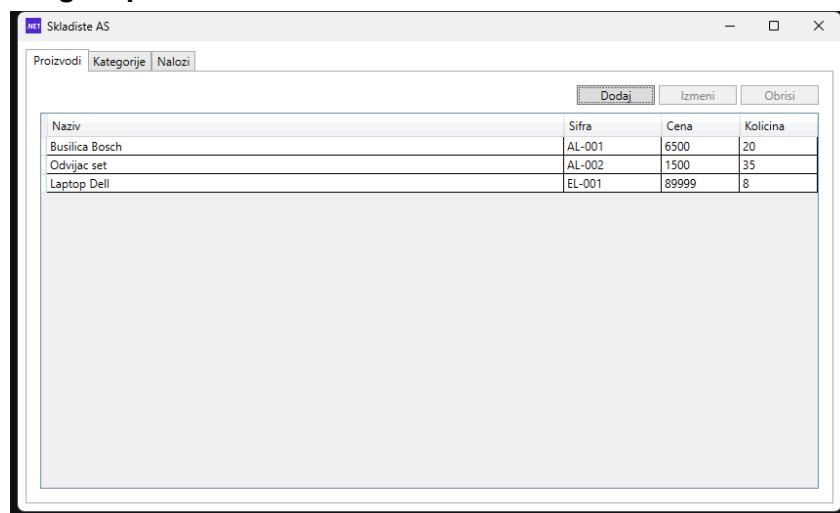
10. Screenshot aplikacije

Prijava korisnika



The screenshot shows a login window titled "Prijava" with a ".NET" icon in the top-left corner and a close button in the top-right. The window contains two text input fields: "Korisnicko ime" (Username) and "Lozinka" (Password). Below these fields is a button labeled "Prijavi se" (Login).

Pregled proizvoda



The screenshot shows a product overview window titled "Skladiste AS". It has a tabbed interface with "Proizvodi" (Products) selected, and other tabs for "Kategorije" (Categories) and "Nalozi" (Orders). Above the table are three buttons: "Dodaj" (Add), "Izmeni" (Edit), and "Obrisi" (Delete). The table lists three products with their names, IDs, prices, and quantities.

Naziv	Sifra	Cena	Kolicina
Busilica Bosch	AL-001	6500	20
Odvijac set	AL-002	1500	35
Laptop Dell	EL-001	89999	8

Kreiranje naloga sa stavkama

NET

Novi nalog

X

Broj naloga

PR-2026-001

Stavke

Laptop Dell

1

Dodaj

Proizvod	Kolicina
Busilica Bosch	2
Laptop Dell	1

Otkazi

Sacuvaj

Spisak naloga

NET

Skladiste AS

-

□

X

Proizvodi

Kategorije

Nalozi

Novi nalog

Broj naloga	Datum	Ukupna vrednost	Broj stavki
PR-2026-001	8/29/2026 6:37:49 PM	102999	2